Software Architecture and Design

CSIT306

Prof. Prajish Prasad

Technical Report

27th October 2025

Members:
Piya Shah
Sneh Pahuja
Varsha Gunturu

Table of Contents

Implementation Summary for all Features

Below is a table documenting which features from our A3 milestone based on the user stories we fully implemented, partially implemented, or did not implement.

Priority	**Total Features**	**Fully Implemented**	**Partially Implemented**	**Not Implemented**
Top	7	4	1	2
Medium	6	5	0	1
Low	2	2	0	0
Total	**15**	**11**	**1**	**3**

Top Priority Features

Feature 1: Multi-File Upload from Device

Status: Partially Implemented (single file only)

API Endpoints:
POST /api/upload/files/
POST /api/upload/bulk/

This feature was implemented using DocumentUploadView. It supports PDF, JPG, and PNG files. It processes one file per request currently to prevent overuse of the OCR API since it has limited credits. It creates Document and ProcessingQueue records and performs OCR using Azure Form Recognizer with a mock OCR fallback in case the API request fails.

Feature 2: Document Categorization and Selection

Status: Fully Implemented

API Endpoints:
POST /api/upload/categorize/

This feature was implemented using DocumentCategorizationView. The Category model supports all seven document types. Batch categorization is not supported currently as batch uploads are not supported. The category selection directly influences OCR extraction logic and Excel formatting.

Feature 3a: Split-Screen Review Layout

Status: Fully Implemented

API Endpoints:
GET /api/documents/{id}/
GET /api/review/

This feature was implemented via DocumentViewSet and DocumentReviewViewSet. The DocumentDetailSerializer returns document metadata, file path, category, and extracted fields with which the frontend renders a split-screen layout.

Feature 3b: Highlighting Low-Confidence Items

Status: Not Implemented

API Endpoints:

The OCR does not return confidence scores for each item due to its format. Thus, to highlight low-confidence items custom logic would have to be built. Due to time constraints, this feature was not implemented.

Feature 3c: In-Table Data Editing

Status: Fully Implemented

API Endpoints:
PATCH /api/review/{id}/update-field/

This feature allows users to directly update extracted fields in the table. When a field value gets the field is marked as manually_verified.The original confidence score still remains for auditing.

Feature 3d: Advanced Image Zoom and Pan

Status: Implemented (Frontend Only)

API Endpoints:
-
The backend provides the document file path. All zoom and pan interactions are handled on the frontend.

Feature 3e: Document-Level Approval Workflow

Status: Fully Implemented

API Endpoints:
POST /api/documents/{id}/finalize/

This feature updates document status to approved or error. Only approved documents are included in Excel exports.

Feature 3f: Progress Tracking and Session Management

Status: Fully Implemented (Manual Review Progress)

API Endpoints:
GET /api/documents/ - List all documents with review status filtering

This feature helps users track their review progress through the document list (BulkReviewList page), which shows all documents and their review status: review_pending (needs human

review), approved (reviewed and finalized), or error. This way they can easily see what documents require their attention, and which documents are already processed.

Feature 3g: Highlighting Unidentifiable Text

Status: Not Implemented

There is no endpoint to explicitly identify unreadable OCR regions. This could potentially be inferred from empty or very low-confidence fields but this feature was not implemented due to time constraints.

Feature 3h: Final Preview Before Approval

Status: Partially Implemented

API Endpoints:
GET /api/documents/{id}/

All extracted data required for preview is already available via the document detail endpoint. There is no need to have a separate dedicated endpoint as the user can always see the preview table.

Feature 4: Formatted Excel Generation and Download

Status: Fully Implemented

API Endpoints:
GET /api/download/excel/?document_ids[]=

This feature was implemented using ExcelDownloadView and excel_generator.py. It supports category-specific formatting, multi-sheet output, and includes only approved documents. A fallback manual Excel upload is also available.

Medium Priority Features

Feature 5: File Management and Organization

Status: Fully Implemented

API Endpoints:
GET /api/documents/?city=&year=&month=&document_type=
GET /api/stats/filter/

Documents can be filtered by city, date, and category. We ensured database indexing for efficient querying.

Feature 6: Processing Status and Queue Management

Status: Not needed

API Endpoints:
GET /api/processing-queue/

Since only one document is sent to the OCR at time, there was no requirement to have a dedicated page to check the OCR progress of multiple documents. Instead the upload page just shows a loading/processing status.

Feature 7: Student Attendance Analytics

Status: Fully Implemented

API Endpoints:
GET /api/analytics/attendance/statistics/ - Returns overall attendance rates (Present vs Absent percentages) for the selected period which is displayed through a set of cards showing Total Students, Average Attendance, and Today's Attendance.

GET /api/analytics/attendance/trend/- Returns day-by-day attendance counts over the last 30 days which is visualised by a line chart.

Feature 8: Student Grades Analytics

Status: Fully Implemented

API Endpoints:

GET /api/analytics/grades/distribution/ -A pie chart showing the percentage breakdown of student grades (e.g., "30% A Grade", "45% B Grade").

Feature 10: Student Enrollment and Growth Tracking

Status: Fully Implemented

API Endpoints:

GET /api/analytics/enrollment/growth/ - A bar chart displaying the steady increase (or decrease) in student enrollments throughout the year.

Feature 11a: Daily Center Activity Monitoring

Status: Fully Implemented

API Endpoints:

GET /api/analytics/activity/summary/- Count of activities by type (e.g., Homework, Class Activity).

GET /api/analytics/activity/recent/- Returns: A list of the most recent activities logs.

Feature 11b: Financial Tracking

Status: Fully Implemented

API Endpoints:

GET /api/analytics/financial/budget-utilization/ - Compare the Total Budget vs Total Expenditure for the current fiscal year displayed in a summary card indicating how much of the allocated budget has been utilized.

GET /api/analytics/financial/expenditure-trend/ - Returns monthly expenditure data to identify high-spending periods visualised in a line chart.

Feature 11c: Requisition Tracking

Status: Fully Implemented

API Endpoints:
/api/analytics/requisition/status/ - Returns the count of store requisitions grouped by their status (Pending, Approved, Fulfilled) visualised in the form of a pie chart.
/api/analytics/requisition/top-items/ Identify the most frequently requested store items
/api/analytics/requisition/vs-received/ Compare quantity requested vs quantity actually received

Low Priority Features

Feature 9: Automated Student Flagging

Status: Fully Implemented

Low-attendance and grade-based flagging is implemented.

Feature 11d / Feature 12: Visitor Analytics and Consolidated Visitor Database

Status: Fully Implemented

The Visitor Trend graph, Purpose Summary chart, and the Visitor Outreach List are all fully implemented.

Additional Implemented Features (Outside Core Features)

Authentication and User Management
POST /api/signup/ - User registration (admin only)
POST /api/login/
GET/PATCH /api/profile/ - User profile management

Development and Debugging Utilities

GET /api/stats/

GET /api/stats/filter/

GET /api/dashboard/stats/

POST /api/documents/{id}/annotated/

GET /api/debug/user/{username}/

Testing

There are 12 backend testing across models, views, OCR Integration and Utility Functions. Front-end testing wasn't focused as much.

```
============================= test session starts
==============================
platform darwin -- Python 3.11.14, pytest-7.4.3, pluggy-1.6.0
django: version: 5.2.7, settings: core.settings
rootdir:
/Users/gunturuvarsha/Downloads/UG4_Sem7/Software_306/repo/UpayScanToE
xcel
configfile: pytest.ini
plugins: Faker-22.0.0, django-4.7.0
collected 12 items

api/tests/test_ingestion_utils.py::test_normalize_name PASSED
[  8%]
api/tests/test_ingestion_utils.py::test_get_or_create_student PASSED
[ 16%]
api/tests/test_models.py::TestModels::test_user_creation PASSED
[ 25%]
api/tests/test_models.py::TestModels::test_center_creation PASSED
[ 33%]
api/tests/test_models.py::TestModels::test_document_defaults PASSED
[ 41%]
api/tests/test_models.py::TestModels::test_extracted_data_linking
PASSED [ 50%]
api/tests/test_ocr_integration.py::TestOCRIntegration::test_upload_tr
iggers_processing PASSED [ 58%]
api/tests/test_ocr_integration.py::TestOCRIntegration::test_finalize_
saves_extracted_data PASSED [ 66%]
api/tests/test_views.py::TestViews::test_signup_permission PASSED
[ 75%]
api/tests/test_views.py::TestViews::test_login PASSED
[ 83%]
api/tests/test_views.py::TestViews::test_document_list_filtering
PASSED  [ 91%]
api/tests/test_ingestion_utils.py::test_parse_attendance_status
PASSED   [100%]

======================== 12 passed, 6 warnings in 4.56s
========================
```

Test Breakdown:

- 2 Integration tests
- 4 Model tests
- 2 OCR Integration tests
- 3 API view tests
- 1 Attendance Parsing Text

Test Organization

The backend tests are organized in the api/tests/ directory with the following structure:

- test_models.py - Database model tests
- test_views.py - API endpoint and authentication tests
- test_ocr_integration.py - OCR workflow integration tests
- test_ingestion_utils.py - Data processing utility tests

All tests use pytest with Django integration and are automatically discovered by the test runner.

1. **Model Tests** (test_models.py)

Tests the Django ORM models and their relationships:

- **User Model Testing**
    - User creation and string representation
    - Default role assignment (employee)
    - User-center relationships
- **Center Model Testing**
    - Center creation and string representation
    - City and name validation
- **Document Model Testing**
    - Document creation with default status (uploaded)
    - File metadata handling (filename, size, type)
    - Document-category relationships
- **ExtractedData Model Testing**
    - Linking extracted data to documents
    - Confidence score validation

    - Validation status defaults (passed)

2. **API View Tests** (test_views.py)

Tests the REST API endpoints and authentication/authorization:

- **Authentication Tests**
    - Login endpoint generates JWT tokens (access & refresh)
    - Token includes user role information
- **Authorization Tests**
    - Only admins can create users (signup endpoint)
    - Employees receive 403 Forbidden when attempting user creation
- **Document Filtering Tests**
    - Employees only see their own documents
    - Admins see all documents across all users
    - Pagination handling

3. **OCR Integration Tests** (test_ocr_integration.py)

Tests the document processing pipeline with mocked OCR services:

- **Upload Processing**
    - File upload triggers processing queue
    - Mock OCR processing to avoid external API calls
    - Document status transitions (uploaded → review_pending)
- **Finalization Workflow**
    - Saving extracted data after user review
    - Document status changes to approved
    - Data ingestion into the database

4. **Utility Function Tests** (test_ingestion_utils.py)

Tests data processing and normalization utilities:

- **Name Normalization**
    - Whitespace trimming
    - Case normalization (uppercase)
    - Special character removal

- **Student Matching Logic**
    - Create new students
    - Match existing students (case-insensitive)
    - Center-specific student isolation
- **Attendance Status Parsing**
    - Parse various attendance formats (P, Present, A, Absent, L, etc.)
    - Handle unknown statuses gracefully

Component	Coverage
Models	User, Center, Document, ExtractedData, Category
Authentication	JWT login, token generation, role-based access
Authorization	Admin-only endpoints, document ownership filtering
File Upload	Document upload, file validation, processing queue
OCR Integration	Mocked OCR processing, data extraction workflow
Data Ingestion	Name normalization, student matching, attendance parsing
API Endpoints	Login, signup, document list, document finalize

Collaboration

Since our entire team is composed of Computer Science minors, our collaboration process was a little different from what is usually expected in a Scrum team, or how more experienced developers work. This was our first time working with Git in a shared codebase and also our first time developing a project of this scale. On top of that, it was our first experience working with React Native and using Django for something more complex. Due to this, a lot of the initial collaboration was more focused on collaborative learning such as figuring out how Git, Django, and React work, how to push and pull changes properly, how to create and manage branches, and what kind of commit messages make sense. This method over having more strict divisions in our tasks ensured everyone was on the same page. We still did keep broad divisions in our tasks. Piya worked on the frontend and training the OCR, Varsha worked on the OCR logic and excel conversion, and Sneh worked on the backend set up and dashboard analytics. All of us worked on integration of the different components.

Due to these considerations, using a very strict Git-based tracking system for features did not work very efficiently for us as it made the process a bit more confusing. Instead, we relied more on communication through meetings and in-person discussions to stay on the same page. Though this approach is not ideal or recommended for more experienced development teams, it made sense for us as a first collaborative experience making a project of this level and helping ensure that everyone understood and was involved in the decisions being made.

We had two kinds of meetings. The first kind was on a lower frequency of every one to two weeks where the meeting time was longer than simple progress updates where it was almost a co-working space with us jointly solving problems, debugging issues, and discussing how certain parts of the system should work. The second kind of meeting was a shorter, simple SCRUM style meeting (we discussed individual updates on what was done, what we were currently working on, and issues we found) which we had two to three times a week, commonly before or after class. We also made it a point to not only discuss challenges we were facing but also positives in either the task progress, or our learnings as well because one person’s learnings always helped the others in some way as well.

As the project progressed, we gradually became more comfortable with Git, React Native, and Django, allowing us to better isolate tasks and work in a way that better represented the processes of a traditional Scrum team. By the end of the project, our collaboration became easier, and if we were to continue working on this project or take on similar ones in the future, we can confidently say that we would be much better prepared to follow more efficient workflows, as in this case shared understanding was at a slightly higher priority.

The Product Owner was Varsha as she handled the majority of the direct communication with our client, though we were all always present for our meetings. The Scrum Master was Sneh ensuring we as a team were on the same page and we were progressing on time with quality.

Client Feedback

Client feedback was taken at different stages of the project through meetings, emails, and shared prototype videos. In the initial stages, the feedback mainly focused on understanding how the team currently works, the types of documents they handle, and the problems they face in their existing scan-to-excel process. These early discussions helped us understand their expectations for the user experience, permissions, nature of documents, filtering, the overall purpose of the dashboard, and most importantly, what to prioritise.

As we shared prototypes and updates, the feedback became more detailed and technical. Deboshree clarified that users should only have access to documents uploaded from their own centre and that admins should be able to view all documents and user activity. Her main priority was the accuracy of the documents, and a system that reduced the manual labour behind converting these documents to excel. She wanted a filter-based document view and mentioned that having upload options including bulk manually-categorised uploads, as well as bulk auto-categorised uploads would be useful but depending on the development time it was alright if not inculcated. We also proposed some additional ideas, some of which she accepted such as the option to add an ‘Upload Excel’ feature where for data that is already stored in excel, it can still be connected to the application so that older data can be integrated for insights in the dashboard as well.

For the dashboard, we discussed different approaches for showing data. One option was using a fixed Excel-style format, while another was having a dynamic dashboard where users could select and filter the data points they wanted to see. Deboshree felt the second option would be more flexible and valuable in the long run. She also helped us understand what analytics and KPIs were required for the dashboard.

The feedback on the presentation day was largely positive. The team found the system clear, simple to use, and easy to understand. They mentioned that it would make their existing workflow much easier, organised and efficient. They also like the simple and easy-to-understand UI design. They did have some questions regarding document accuracy and whether additional training would be required for different document types. We explained that the model had been trained using the limited sample documents provided, and that accuracy could be improved with more samples. We also clarified that if completely new document formats were introduced, additional training would be needed and the time required to train each document would be 1-2 hours.

Overall, the client was satisfied with the solution and the direction of the project. Their feedback helped place a clear direction of how the project should be like for both the functionality and the design. We were pleased to hear that they would like to bring the application to UPAY.

Implementation

Github url: https://github.com/snehpahuja/UpayScanToExcel.git

Technologies and tools used

Frontend Technologies

React (JavaScript Framework)

React was used to develop the frontend as a single-page application (SPA). A component-based architecture was adopted to modularise the user interface into reusable units such as

authentication pages, document upload views, dashboards, and review screens. This approach improved maintainability and supported incremental feature development.

Although the application is implemented as a **Single Page Application (SPA)** using React, it is **not a single-page system in terms of functionality or user interaction**.

- The browser loads one root HTML file.
- Multiple **logical pages** exist (Login, Upload, Dashboard, Review, Export).
- Navigation between these pages is handled using **client-side routing**.
- Each page corresponds to a distinct React route and component tree.

Vite (Build Tool and Development Server)

Vite was selected as the frontend build tool due to its fast development server, efficient module replacement, and minimal configuration requirements. This significantly reduced development iteration time compared to traditional bundlers.

React Router

Client-side routing was implemented using React Router, enabling seamless navigation between application views without full page reloads. This was essential for maintaining session state and ensuring a smooth user experience.

HTML5 and CSS3

HTML5 and CSS3 were used for structural layout and styling. Initial static HTML prototypes were progressively refactored into React components while preserving the original design intent.

Fetch API

The browser Fetch API was used for asynchronous communication with the backend REST endpoints, including authenticated requests for file uploads, document retrieval, and processing status updates.

Backend Technologies

Python 3

Python was chosen for backend development due to its strong ecosystem for web development and machine learning integration.

Django Framework

Django served as the core backend framework, providing URL routing, request handling, ORM-based database interaction, and security features. Its structured project layout enabled clear separation of concerns between models, views, and configuration.

Django REST Framework (DRF)

DRF was used to expose backend functionality as RESTful APIs. These APIs handle user authentication, document uploads, document metadata management, and OCR processing workflows.

JWT Authentication (SimpleJWT)

JSON Web Token (JWT) authentication was implemented to enable secure communication between the frontend and backend. Tokens are issued upon successful login and attached to subsequent API requests.

Database (SQLite – Development)

SQLite was used during development due to its simplicity and ease of setup. The data model supports users, uploaded documents, processing status, and extracted metadata. The system was designed to allow migration to PostgreSQL for production deployment.

File and Media Handling

Django's media handling mechanisms were used to store uploaded documents (PDFs and images) and serve them securely for review and processing during development.

OCR and Document Intelligence

Microsoft Azure Document Intelligence (formerly Form Recognizer)

The application integrates Microsoft Azure Document Intelligence to perform Optical Character Recognition (OCR) and structured data extraction from uploaded documents.

Custom Model Training

A custom document intelligence model was trained using labelled data provided by the partner company. Transfer learning techniques were employed, where the base pretrained Azure model was retained while the final layers were fine-tuned on upay-specific documents. This approach improved extraction accuracy for company-specific document formats such as forms, invoices, and attendance records.

Processing Pipeline

1. Documents are uploaded via the frontend.
2. The backend stores the document and queues it for processing.
3. The document is sent to Azure Document Intelligence for analysis.
4. Extracted text and structured fields are returned and stored.
5. Results are made available to the user for review and export.

During early development, a mock OCR processor was used to validate the end-to-end pipeline before full Azure integration.

Development and Supporting Tools

1. **Git and GitHub** – Version control, collaboration, and change tracking.
2. **Visual Studio Code** – Primary integrated development environment.
3. **Python Virtual Environment (venv)** – Dependency isolation for backend packages.
4. **Postman and Browser Developer Tools** – API testing, request inspection, and debugging.
5. **Node.js and npm** – Frontend dependency management.

Deployment details

Deployment Architecture (Planned)

The system was designed for deployment using a cloud-based architecture with separation between frontend, backend, and AI services.

- **Frontend**: Built as static assets using Vite and served via a static hosting service.
- **Backend**: Deployed as a Django web service exposing REST APIs.
- **Database**: Planned migration from SQLite to PostgreSQL for production.
- **OCR Service**: Microsoft Azure Document Intelligence accessed via secure API endpoints.

Cloud Platform

Render (Planned Deployment Platform)

Render was selected as the intended deployment platform due to its support for Django web services, static frontend hosting, and environment variable management.

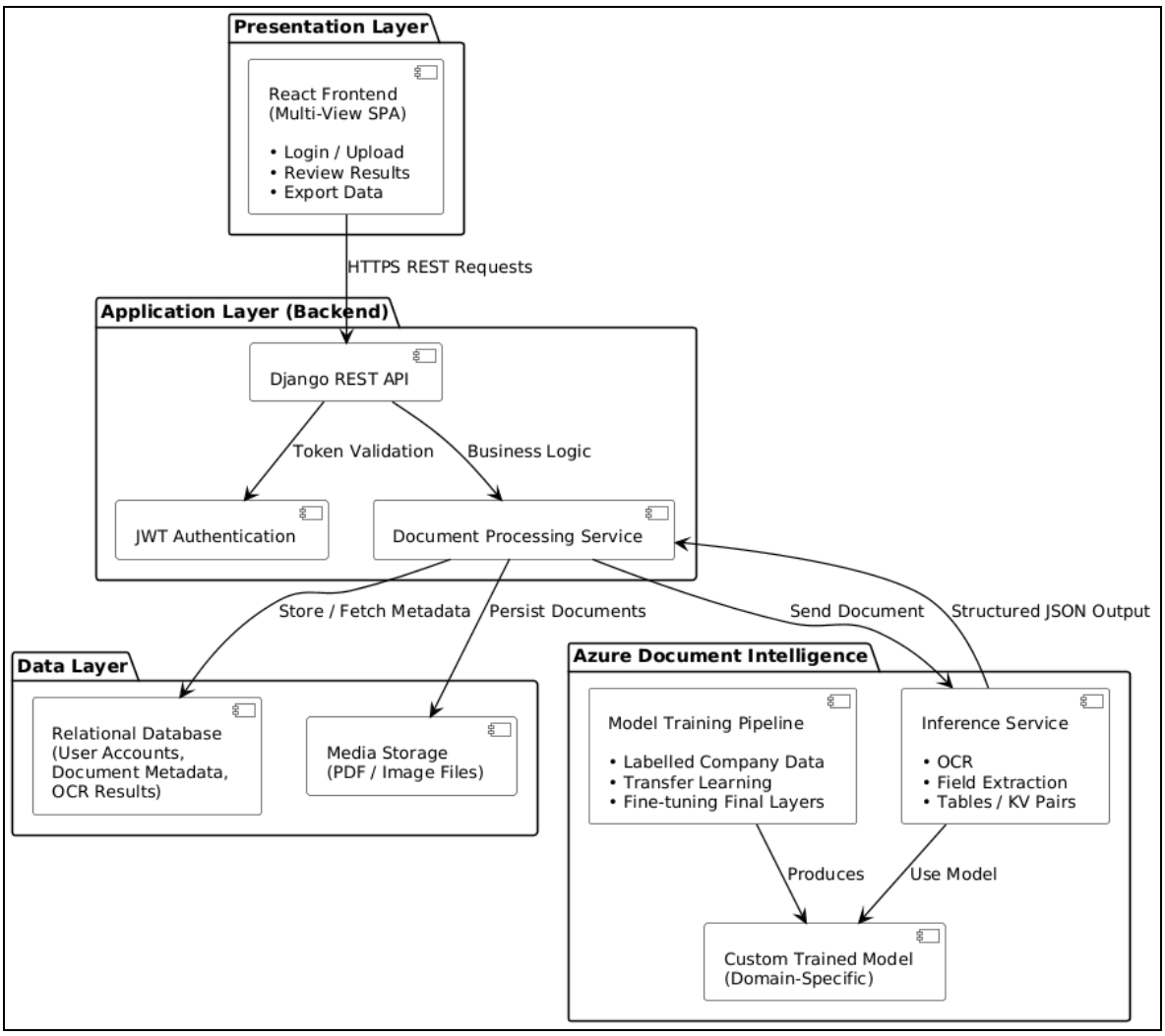
Environment Configuration

The deployment design relies on environment variables for sensitive configuration, including:

- Django SECRET_KEY
- DEBUG flag
- Database connection string
- JWT configuration
- Azure credentials (endpoint URL and API key)
- Azure Document Intelligence model ID

Current Deployment Status

- The application runs successfully in a local development environment.
- Final deployment to Render was not completed before submission.
- OCR integration with Microsoft Azure Document Intelligence is functionally implemented and tested locally.
- The system architecture supports production deployment with minimal configuration changes.



Architecture Diagram

Presentation Layer (Frontend)

The presentation layer is implemented using React and executes entirely within the user's web browser. It is responsible for all user-facing interactions and presentation logic.

Responsibilities:

- User authentication (login and logout)
- Document upload interface
- Display of uploaded documents
- Review of extracted OCR results

- Triggering data export operations

Key characteristics:

- Implements client-side routing to render multiple application views
- Communicates exclusively with backend services via RESTful APIs
- Does not directly access the database or OCR services
- Maintains authentication state using JWT tokens

All communication between the frontend and backend occurs over HTTP/HTTPS using REST-based endpoints.

Application Layer (Backend – Django REST)

The backend serves as the central orchestration layer of the system and mediates all interactions between the frontend, data storage, and external AI services.

Responsibilities:

- User authentication and authorisation
- Validation and secure storage of uploaded documents
- Management of document metadata and processing states
- Secure communication with Azure Document Intelligence
- Exposure of REST APIs to the frontend

Key components:

- Django for request handling, routing, and configuration
- Django REST Framework for REST API construction
- JWT-based authentication for stateless security
- ORM-based database access for persistent storage

The backend ensures that:

- Sensitive credentials (e.g., Azure API keys and secrets) are never exposed to the frontend
- OCR processing logic is abstracted behind a dedicated service layer

- The frontend remains fully decoupled from OCR implementation details

Intelligence and Processing Layer (Azure Document Intelligence)

Microsoft Azure Document Intelligence is used as an external AI service for Optical Character Recognition (OCR) and structured data extraction.

Responsibilities:

- Text recognition from scanned documents and PDFs
- Extraction of structured information such as tables and key–value pairs
- Domain-specific interpretation of document layouts

Custom Model Training

- A pretrained Azure base model was utilised as the foundation.
- The final layers were fine-tuned using labelled documents provided by the company.
- This transfer learning approach improves extraction accuracy for domain-specific document layouts.
- Training data consisted of real-world documents used by the organisation.

Processing flow:

1. The backend sends the uploaded document to the Azure service.
2. The custom-trained model performs OCR and structured data extraction.
3. Results are returned in structured JSON format.
4. The backend stores the processed data.
5. The frontend retrieves and displays the results to the user.

Data Flow Summary

End-to-end workflow:

1. The user uploads a document via the frontend.
2. The frontend sends the file to the backend API.

3. The backend stores the document and associated metadata.
4. The backend invokes Azure Document Intelligence for processing.
5. OCR results are returned and persisted.
6. The frontend fetches the processed results for review and export.

This design ensures:

- Clear separation of concerns across system layers
- Secure handling of AI credentials and sensitive data
- Replaceability of the OCR engine without impacting frontend logic

Instructions to run your application.

System Requirements

- Windows 10 or later
- Python 3.10 or higher
- Node.js 18 or higher (includes npm)
- Git
- Microsoft Azure account with:
    - Azure Document Intelligence endpoint
    - API key
    - Custom trained model ID

Backend Setup (Django REST)

1. Clone the backend repository using:

   git clone https://github.com/snehpahuja/UpayScanToExcel.git

2. Navigate to the backend directory using: cd backend
3. Create a virtual environment using: python -m venv venv
4. Activate the virtual environment using: venv\Scripts\Activate
5. Install dependencies using: pip install -r requirements.txt

6. Set required environment variables (SECRET_KEY, DEBUG, AZURE_ENDPOINT, AZURE_KEY, AZURE_MODEL_ID).
7. Apply database migrations using: python manage.py migrate
8. Generate sample data by running python api/create_sample_data_extended.py. (This step is required for analytics and dashboard charts)
9. Start the backend server using: python manage.py runserver
10. Log in using
    a. Username: admin
    b. Password: adminpass

The backend running link will be highlighted in the terminal as http:localhost:<link>

Frontend Setup (React + Vite)

1. Navigate to the frontend directory using: cd frontend
2. Install dependencies using: npm install
3. Start the development server using: npm run dev

The frontend will be highlighted in the shell where frontend is running as http:localhost:<link>

Running the Application

- Open the frontend URL in a browser.
- Log in or register a user account.
- Upload documents through the upload interface.
- Documents are processed by Azure Document Intelligence using the custom-trained model.
- Extracted data is available for review and export.

Notes

- Backend and frontend servers must be running simultaneously.
- Azure credentials are required for OCR functionality.

- The application was tested in a local development environment; cloud deployment was planned but not completed.

Demo video

Url link:
https://drive.google.com/file/d/1xOOfG93s54r_IA0dfYmihgy8lv3VDzyw/view?usp=sharing

Final reflections

Piya shah

I learnt a lot about frontend development during the course of this project. I worked on developing the entire frontend UI/UX flow. Being a CS minor, I had not taken any course where I could learn about the UI/UX features of such dashboards. I started with developing multiple html files to make a working prototype of the dashboard. This included the index file and individual files for all the other pages. However, I had never worked with react before, hence used GenAI to help me convert those files into react components. I finally understood the importance of github and version control. For me, the most important part was understanding that there are so many things to consider to make a good UX flow for such a dashboard. Although, it was challenging to work with React for the first time; A lot of errors that came up were unfamiliar to me. I went through a lot of tutorials to understand the components and their connections. I now am confident to work with React again.

Sneh Pahuja

My most valuable learning in this project was being able to understand what to prioritize, and how to collaborate with others in an efficient way. I can be over ambitious sometimes, and in this project I had to learn how to ensure the client is satisfied and the key goals are met without over extending ourselves and promising something that eventually falls through due to time or other constraints. Another valuable thing I learned was how important communication with both the client and the team members is and how it gives the project direction and structure. I

also learned many technical skills during this project, given that I had only built a web application once before and it was nowhere as complex as this project was. This was a project that I had to build from the ground up, first figuring out the problem and its specifications and then ideating, designing, implementing and testing. Working with git is also something I learned during this. It was a bit confusing navigating the merges, and going back to the commits, but by the end we all felt more comfortable and confident , and understood how valuable it is in projects like these.

Varsha Gunturu

I learnt to collaborate on such a big project for the first time. Learning most of the technical details and implementations along the way. This included Django, OCR, React, proper API and authentication. I believe I understand why github is so important. Especially how easy it was to check out on the progress of each of us, where we created our own branches and implemented changes one by one. We were not only learning new concepts like clean coding, modules, controllers, but also were implementing it one by one. I believe experiencing structured meetings and progress gave us a deeper understanding and idea of how huge projects run in software development in most big companies. Since I never had any software development internship, this project surely opened many opportunities for me. Because now I do have the confidence that I can work with a project on a large scale. Some challenges I did face was figuring out the OCR. I did try multiple github projects, api along with Sneh to figure out which one gave the best results/accuracy. When working with github properly with a team, I did face difficulty in merging due to conflicts, but it took me time to understand. I believe version control also, to some extent, helped me explore thoroughly cause I know if there is some error I can always resort to the last working version.